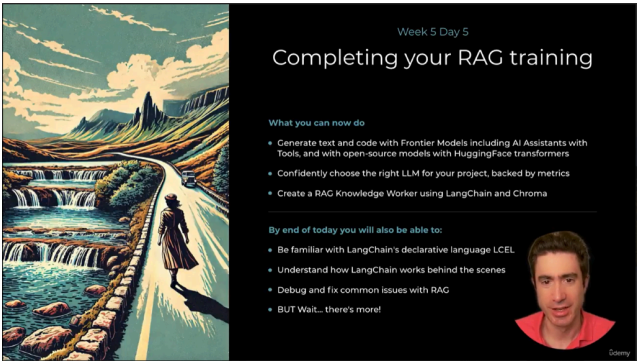
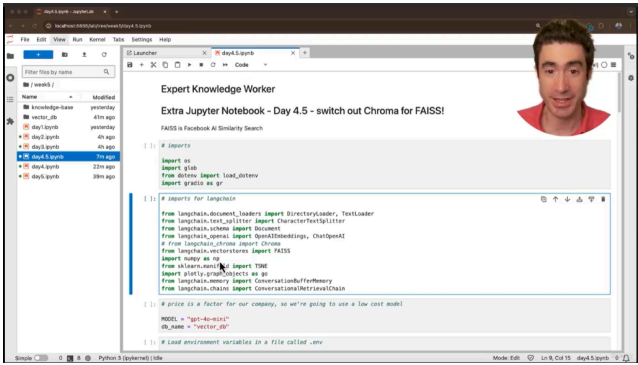




- **Hoàn thiện kiến thức về RAG:** Bạn sẽ chuyển từ chuyên gia RAG thành bậc thầy RAG.
- **Ngôn ngữ biểu thức LangChain (LCEL):** Bạn sẽ được giới thiệu về LCEL, ngôn ngữ khai báo để thiết lập các chuỗi trong LangChain.
- **Cách LangChain hoạt động:** Bạn sẽ tìm hiểu về cách LangChain hoạt động bên trong.
- **Gỡ lỗi và khắc phục sự cố RAG:** Bạn sẽ học cách chẩn đoán và khắc phục các sự cố thường gặp với RAG.
- **Thử nghiệm với cơ sở dữ liệu vector khác:** Bạn sẽ được xem một ví dụ về việc sử dụng một cơ sở dữ liệu vector khác trong JupyterLab.



Chuyển đổi vector data store từ Chroma sang FAISS:

- Bạn sẽ được hướng dẫn cách chuyển đổi vector data store từ Chroma sang FAISS (Facebook AI Similarity Search) một cách dễ dàng.
- FAISS là một thư viện tìm kiếm tương đồng mã nguồn mở phổ biến do Facebook phát triển.

FAISS là gì:

- FAISS là một thư viện cho phép tìm kiếm nhanh chóng các vector gần với các vector khác.
- Nó thường được sử dụng như một loại vector store trong bộ nhớ (in-memory).
- FAISS không lưu trữ dữ liệu trên đĩa, nhưng nó có thể được sử dụng để lưu trữ và truy xuất các vector tương tự.
- FAISS cũng có thể được sử dụng như một thư viện để thực hiện tìm kiếm tương đồng.

LangChain hỗ trợ FAISS:

- LangChain có các khái niệm trừu tượng được xây dựng xung quanh FAISS, cho phép nó được sử dụng thay thế cho Chroma.

Sự khác biệt giữa Chroma và FAISS:

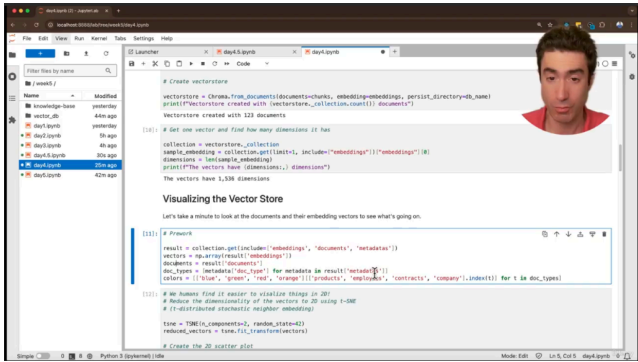
- Chroma là một vector store lâu dài, lưu trữ dữ liệu trên đĩa.
- FAISS là một vector store trong bộ nhớ, không lưu trữ dữ liệu trên đĩa.

Cài đặt FAISS:

- FAISS có hai biến thể: CPU và GPU.
- Bạn cần cài đặt biến thể phù hợp với môi trường của mình bằng pip.
- Phiên bản GPU mang lại hiệu suất cao hơn cho các dự án đòi hỏi nhiều hiệu năng.

Mã tương tự:

- Mã để sử dụng FAISS gần giống với mã để sử dụng Chroma, với một vài thay đổi nhỏ.
- Trong đoạn mã này, import Chroma bị comment và thay thế bằng import FAISS.



Thay đổi duy nhất trong mã:

- Dòng mã tạo vector store đã thay đổi từ chroma.from_documents thành vice.from_documents.
- Điều này cho thấy sự dễ dàng trong việc thay thế vector data store trong LangChain.

Sự khác biệt về tham số:

- vice.from_documents không nhận tham số persist_directory vì FAISS không lưu trữ dữ liệu trên đĩa.

Cách truy vấn dữ liệu từ FAISS:

- Các dòng mã truy vấn dữ liệu từ FAISS khác với Chroma.
- Bạn cần biết cách truy vấn số lượng vector và số chiều vector từ FAISS.

Số chiều vector:

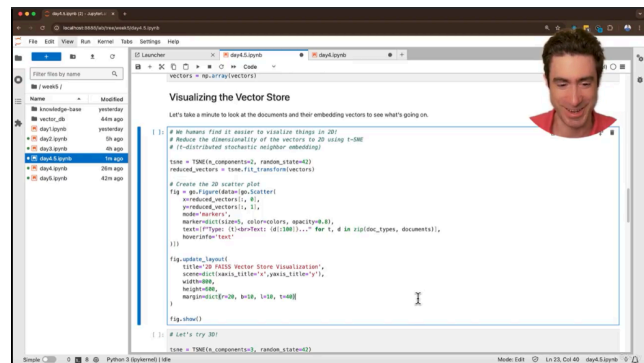
- Số chiều vector vẫn giữ nguyên vì sử dụng cùng một mô hình OpenAI embeddings.

Phản tiền xử lý (pre-work):

- Phản tiền xử lý để lấy dữ liệu vector, document và doc type khác nhau giữa Chroma và FAISS.
- Bạn sẽ thấy cách truy vấn dữ liệu từ Chroma và FAISS.
- Mã được cung cấp để lấy dữ liệu cần thiết từ FAISS và ánh xạ màu sắc cho biểu đồ Plotly.

Mục tiêu:

- Cho thấy sự dễ dàng trong việc thay thế vector data store.
- Minh họa cách sử dụng FAISS thay vì Chroma.
- So sánh cách truy vấn dữ liệu từ Chroma và FAISS.



Trực quan hóa vector store FAISS:

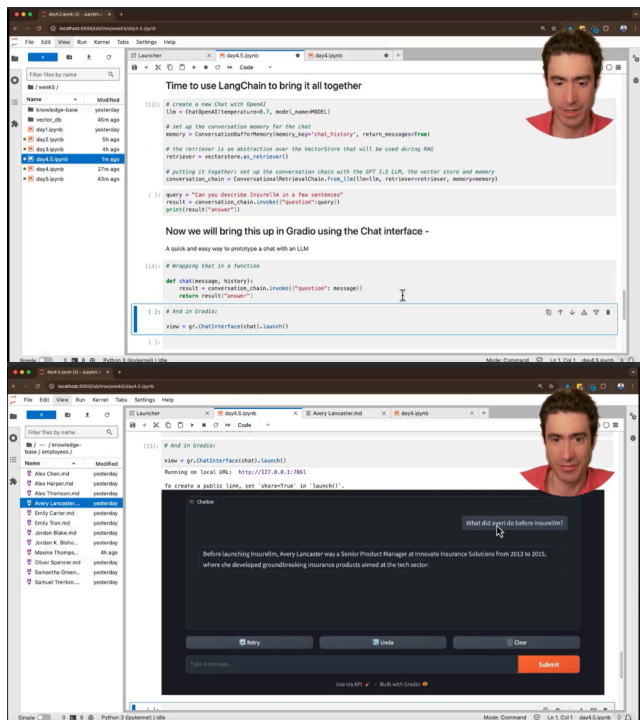
- Mã để trực quan hóa vector store (FAISS) gần giống hệt với mã trực quan hóa vector store Chroma.
- Sự khác biệt duy nhất là tiêu đề của biểu đồ, được thay đổi từ "Chroma" thành "FAISS".
- Biểu đồ 2D và 3D được tạo ra để hiển thị các vector trong không gian.

Kết quả trực quan hóa:

- Kết quả trực quan hóa cho thấy các vector được phân tách tương tự như khi sử dụng Chroma.
- Điều này là do cùng một phương pháp vector hóa được sử dụng.
- Biểu đồ 3D cho thấy sự phân tách rõ ràng hơn giữa các nhóm vector.

Điểm quan trọng:

- Đoạn trích nhấn mạnh sự dễ dàng trong việc thay thế vector store trong LangChain.
- Mã gần như giống hệt nhau, cho thấy tính trừu tượng của LangChain.
- FAISS là một lựa chọn thay thế khả thi cho Chroma.



Thay thế vector store:

- Mã để tích hợp FAISS vào LangChain gần như giống hệt mã tích hợp Chroma.
- Điều này cho thấy sự dễ dàng trong việc thay thế vector store trong LangChain.

Khả năng xử lý lỗi chính tả:

- Ứng dụng RAG có khả năng xử lý lỗi chính tả và vẫn tìm kiếm được thông tin chính xác.
- Ví dụ, khi hỏi "What did Avery do before?" với lỗi chính tả "Avery", ứng dụng vẫn tìm được thông tin về Avery Lancaster.

Truy xuất thông tin chính xác:

- Ứng dụng RAG có thể truy xuất thông tin chính xác từ cơ sở dữ liệu vector, ngay cả khi câu hỏi có lỗi chính tả hoặc không sử dụng tên đầy đủ.
- Điều này cho thấy khả năng hiểu ngữ nghĩa của mô hình.

Sử dụng vector lookup:

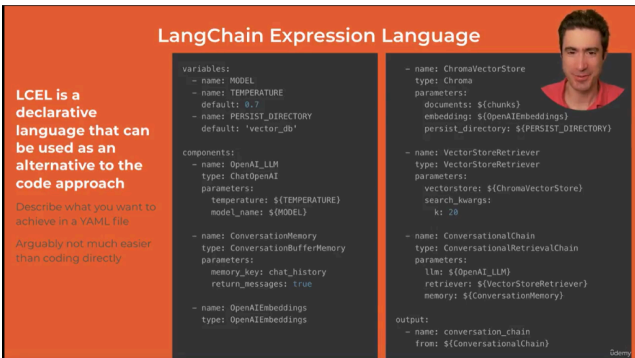
- Vector lookup hiệu quả hơn nhiều so với phương pháp tìm kiếm văn bản thô sơ.
- Vector lookup cho phép tìm kiếm các vector tương tự, ngay cả khi có lỗi chính tả hoặc cách diễn đạt khác nhau.

Thử nghiệm với Chroma:

- Khuyến khích thử nghiệm với Chroma để xác nhận rằng nó cũng hoạt động tương tự.

Tính linh hoạt của LangChain:

- LangChain cho phép dễ dàng thay thế các vector data store khác nhau mà không cần thay đổi nhiều mã.
- Điều này mang lại tính linh hoạt cao cho việc xây dựng các ứng dụng RAG.



Ngôn ngữ biểu thức LangChain (LCEL):

- LCEL là một ngôn ngữ khai báo để thiết lập các chuỗi trong LangChain.
- Nó cho phép bạn mô tả các bước trong pipeline của mình một cách rõ ràng bằng cách sử dụng tệp YAML.

Mục đích của LCEL:

- LCEL cung cấp một cách thay thế cho việc viết mã Python trực tiếp để xây dựng các chuỗi LangChain.
- Nó có thể giúp bạn dễ dàng hơn trong việc quản lý và hiểu các pipeline phức tạp.

Cấu trúc tệp YAML:

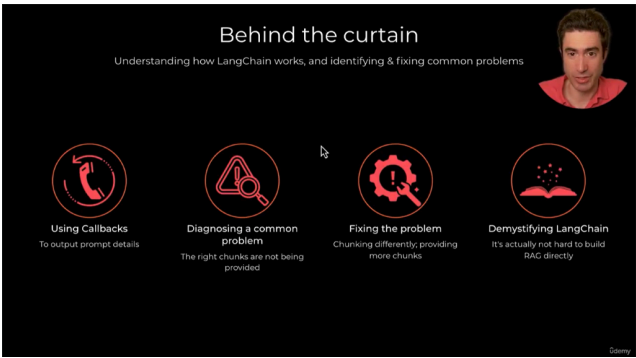
- Tệp YAML LCEL định nghĩa các biến, thành phần và đầu ra của chuỗi.
- Các thành phần bao gồm LLM, bộ nhớ, embeddings, vector store, retriever và chuỗi.
- Cấu trúc này tương ứng chặt chẽ với mã Python được sử dụng để xây dựng các chuỗi LangChain.

Ưu điểm và nhược điểm của LCEL:

- LCEL có thể hữu ích cho những người thích cách tiếp cận khai báo.
- Tuy nhiên, người nói thích sử dụng mã Python trực tiếp để xây dựng các quy trình làm việc.

Mục đích của việc giới thiệu LCEL:

- Để bạn làm quen với LCEL trong trường hợp bạn gặp nó trong các dự án khác.
- Để bạn hiểu cách LCEL ảnh hưởng tới mã Python.



Cách LangChain hoạt động:

- LangChain không thực hiện quá nhiều phép thuật phức tạp.
- Nó chủ yếu là gọi các hàm thích hợp từ các thành phần cơ bản như Chroma hoặc FAISS.
- Nó truy xuất các tài liệu liên quan và kết hợp chúng vào prompt.

Sử dụng callbacks:

- Bạn sẽ học cách sử dụng callbacks để lấy thông tin về prompt mà LangChain gửi đến OpenAI.
- Điều này giúp bạn hiểu rõ hơn về những gì đang diễn ra bên trong LangChain.

Chẩn đoán vấn đề phổ biến:

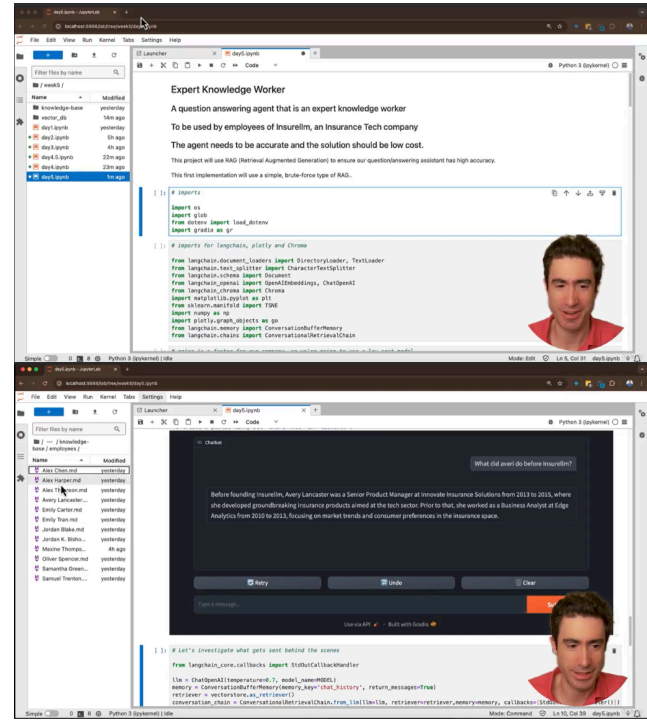
- Bạn sẽ học cách chẩn đoán vấn đề khi các chunk không chính xác được gửi đến mô hình.
- Điều này có thể dẫn đến việc mô hình không cung cấp câu trả lời mong muốn.

Khắc phục vấn đề:

- Bạn sẽ học cách khắc phục vấn đề bằng cách thay đổi cách chia chunk hoặc cung cấp nhiều chunk hơn.

Làm rõ về LangChain:

- Bạn sẽ hiểu rằng LangChain không quá phức tạp và việc xây dựng RAG trực tiếp không quá khó khăn.



Kiểm tra lại khả năng xử lý lỗi chính tả:

- Thử lại câu hỏi "What did Avery do before ensure?" với lỗi chính tả "aviary" để đảm bảo rằng Chroma vẫn hoạt động tốt.

Thử nghiệm với thông tin cụ thể:

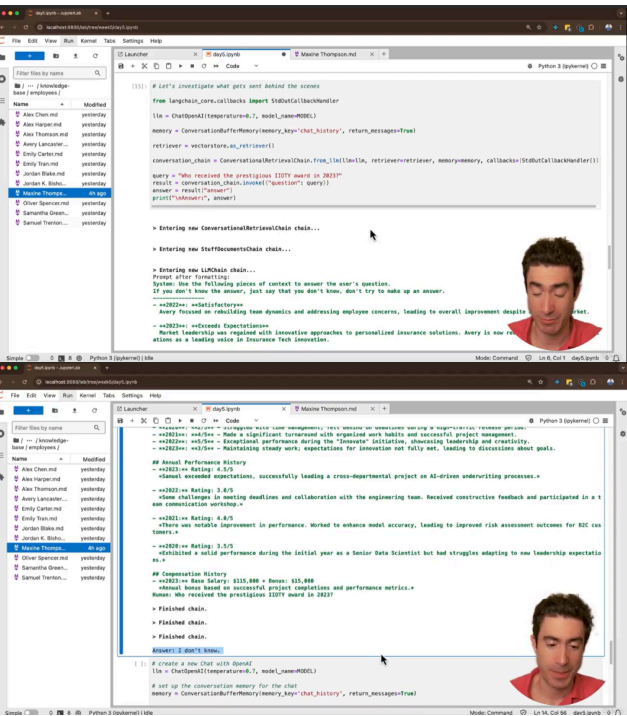
- Xem xét hồ sơ nhân viên của Maxine Thompson, trong đó có thông tin về giải thưởng "Ensure Elm Innovator of the year" mà cô ấy nhận được.
- Đặt câu hỏi "Who received the prestigious Ensure Elm Innovator of the year award in 2023?" để xem liệu mô hình có thể truy xuất thông tin chính xác hay không.

Kết quả không mong muốn:

- Mô hình trả lời "I don't know," cho thấy nó không thể truy xuất thông tin về giải thưởng của Maxine Thompson.

Chẩn đoán vấn đề:

- Bắt đầu quá trình chẩn đoán để tìm hiểu lý do tại sao mô hình không thể truy xuất thông tin chính xác.
- Tìm hiểu về cách LangChain hoạt động bên trong để hiểu rõ hơn về vấn đề.



Sử dụng Standard Out Callback Handler:

- Bạn sẽ được giới thiệu về StandardOutCallbackHandler, một công cụ hữu ích để in thông tin về quá trình hoạt động của LangChain ra đầu ra chuẩn.
- Công cụ này giúp bạn theo dõi những gì đang diễn ra "đằng sau hậu trường" khi LangChain thực thi.

Hiểu cách LangChain hoạt động:

- Bạn sẽ thấy rằng LangChain sử dụng các "chuỗi" (chains) để kết nối các bước khác nhau trong quá trình xử lý.
- Bạn có thể sử dụng các callback khác nhau để in chi tiết về từng bước nếu cần.

Xem Prompt gửi đến GPT-4:

- Bạn sẽ thấy prompt được gửi đến GPT-4, bao gồm hướng dẫn hệ thống và ngữ cảnh được cung cấp.
- Prompt này được thiết kế cẩn thận bởi các chuyên gia LangChain để tối ưu hóa hiệu suất của LLM.
- Prompt này ngăn GPT-4 đưa ra các thông tin sai lệch.

Phân tích ngữ cảnh được cung cấp:

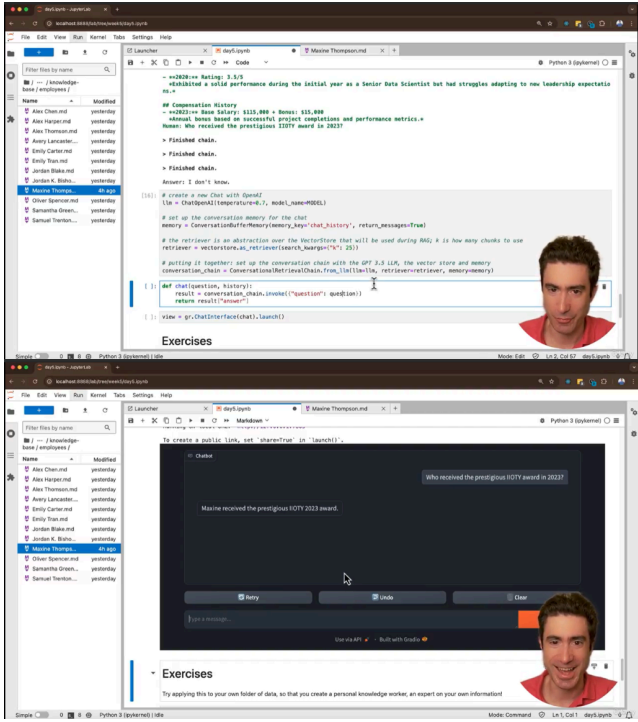
- Bạn sẽ thấy rằng ngữ cảnh được cung cấp cho GPT-4 không chứa thông tin về giải thưởng IOT.
- Điều này giải thích tại sao mô hình trả lời "I don't know."

Chẩn đoán và khắc phục vấn đề:

- Học được rằng việc cung cấp ngữ cảnh không chính xác là một vấn đề phổ biến trong RAG.
- Giới thiệu về một số chiến lược để khắc phục vấn đề này, bao gồm:
 - Xem xét lại chiến lược chia chunk.
 - Gửi toàn bộ tài liệu làm ngữ cảnh.
 - Chia chunk nhỏ hơn.
 - Điều chỉnh độ chồng chéo giữa các chunk.
 - Điều chỉnh số lượng chunk được gửi.

Điều chỉnh số lượng chunk:

- Trong trường hợp này, bạn sẽ học cách thay đổi số lượng chunk được gửi đến GPT-4 để cải thiện kết quả.



Điều chỉnh số lượng chunk:

- Bạn có thể kiểm soát số lượng chunk được gửi đến LLM bằng cách đặt tham số `search_kwargs` trong retriever.
- Trong trường hợp này, số lượng chunk được tăng lên 25.

Nguyên tắc chung về ngữ cảnh:

- Nói chung, việc cung cấp nhiều ngữ cảnh hơn cho LLM là tốt, vì LLM có khả năng tập trung vào ngữ cảnh liên quan và bỏ qua ngữ cảnh không liên quan.
- Tuy nhiên, có một số trường hợp ngoại lệ, chẳng hạn như khi sử dụng các mô hình mới nhất của OpenAI, nơi việc cung cấp quá nhiều ngữ cảnh có thể làm chậm và gây xao nhãng mô hình.

Kết quả cải thiện:

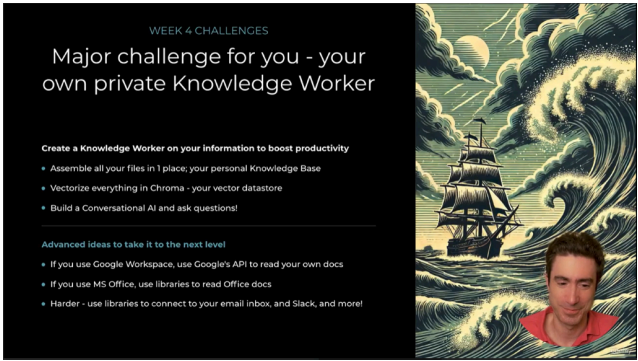
- Sau khi tăng số lượng chunk, mô hình có thể trả lời chính xác câu hỏi về người nhận giải thưởng IOT.

Bài tập:

- Khuyến khích thử nghiệm với các câu hỏi khó, chèn thông tin tùy chỉnh vào tài liệu và thử các chiến lược chia chunk khác nhau.
- Mục tiêu là để hiểu rõ hơn về tác động của các chiến lược chia chunk khác nhau đối với chất lượng kết quả và tìm ra cách cung cấp ngữ cảnh hiệu quả nhất.
- Thay đổi khuyến khích thử nghiệm để tìm ra các tác động của việc cung cấp quá nhiều ngữ cảnh.

Mục tiêu:

- Hiểu cách điều chỉnh số lượng chunk để cải thiện kết quả.
- Nắm vững nguyên tắc chung về cung cấp ngữ cảnh cho LLM.
- Thực hành thử nghiệm với các chiến lược chia chunk khác nhau.



Thử thách cuối tuần:

- Thử thách để người học tự xây dựng "Knowledge Worker" cá nhân của mình.
- Mục tiêu là tăng năng suất cá nhân bằng cách tạo một hệ thống hỏi đáp dựa trên thông tin của chính người học.

Các bước thực hiện:

- Tập hợp tất cả các tệp thông tin vào một thư mục (knowledge base cá nhân).
- Vector hóa tất cả thông tin bằng Chroma (vector data store cá nhân).
- Xây dựng một AI đàm thoại để đặt câu hỏi về thông tin.

Các ý tưởng nâng cao:

- Kết nối với Gmail API để đọc email và vector hóa chúng.
- Sử dụng thư viện Python để đọc các tệp Microsoft Office.
- Sử dụng Google API để đọc các tài liệu Google Drive.

Vấn đề bảo mật:

- Người nói đề cập đến lo ngại về việc sử dụng OpenAI embeddings với dữ liệu cá nhân.
- Đề xuất sử dụng mô hình mã nguồn mở như Bert để vector hóa dữ liệu cục bộ.
 - Sử dụng Google Colab với Google Drive được ánh xạ.
 - Sử dụng llama.cpp để chạy mô hình suy luận cục bộ.

Yêu cầu tối thiểu:

- Người học nên thử nghiệm với ít nhất một vài tài liệu văn bản trong một thư mục để hiểu cách thức hoạt động.

Khuyến khích:

- Người nói hy vọng người học sẽ thực hiện toàn bộ dự án và cạnh tranh với anh ấy.
- Anh ấy sẽ viết một bài đăng trên blog về dự án của mình.

Hoàn thành tuần học về RAG:

- Bạn đã hoàn thành tuần học về RAG (Retrieval-Augmented Generation).
- Bạn đã đạt được 62.5% tiến trình trên hành trình làm chủ LLM (Large Language Model).

Các kỹ năng đã đạt được:

- Bạn đã có kiến thức sâu sắc về cách RAG hoạt động và tại sao nó hiệu quả.
- Bạn hiểu về vector embedding và vector data stores.
- Bạn có thể làm việc với các mô hình tiên tiến, trợ lý AI và công cụ.
- Bạn có thể sử dụng Hugging Face cho các mô hình mã nguồn mở, pipelines, tokenizers và models.
- Bạn có thể lựa chọn LLM phù hợp dựa trên các bảng xếp hạng như Open LLM Leaderboard từ Hugging Face.

Tuần học tiếp theo:

- Bạn sẽ bắt đầu một dự án thương mại mới.
- Bạn sẽ tải xuống một tập dữ liệu từ Hugging Face Hub.
- Bạn sẽ kiểm tra và sắp xếp tập dữ liệu.
- Bạn sẽ xác định các tiêu chí đánh giá thành công.
- Bạn sẽ chuyển từ thể giới suy luận (inference) sang thể giới huấn luyện (training).

